

AI-Based Dataset Hypothesis Recommendation System

Beyza Karagöz

Dept. of Computer Engineering

Kastamonu University

<https://orcid.org/0009-0007-9608-4681>

Abstract—This paper presents the design and implementation of an AI-supported academic performance analysis and hypothesis generation system. The system accepts user-provided academic performance indicators—specifically weekly study hours, absence count, and examination scores—and processes these inputs through a multi-stage automated workflow to produce meaningful academic hypotheses, confidence scores, performance categories, and personalized AI-driven recommendations. The workflow pipeline is implemented on the n8n open-source automation platform and communicates with a modern single-page web application via a RESTful webhook API. Experimental evaluation on simulated student profiles demonstrates a category classification accuracy of 93.3%, a mean confidence score of 82.9, and an average end-to-end webhook response latency of 1.22 seconds, indicating strong suitability for real-time educational decision-support applications.

Keywords—artificial intelligence; hypothesis recommendation; academic performance analysis; n8n workflow automation; webhook API; feature analysis; confidence score; educational decision support

I. INTRODUCTION

Academic performance prediction and monitoring have become central topics at the intersection of educational data mining and learning analytics. Traditional evaluation methods—predominantly linear regression and rule-based grading systems—cannot fully capture the complex, nonlinear relationships present in student behavior data [1]. The rapid evolution of large language models (LLMs) and workflow automation platforms has opened new avenues for building intelligent, automated, and interpretable decision-support systems in education [2].

This paper presents the AI-Based Dataset Hypothesis Recommendation System, a platform that collects three core academic performance indicators (weekly study hours, absence count, and examination score) and processes them through a ten-stage n8n workflow to automatically produce natural-language academic hypotheses, performance categories, confidence scores, and personalized recommendations. The system couples a browser-based dark-themed single-page application with a webhook API, enabling real-time interaction without requiring specialized client software.

The principal contributions of this work are: (1) a domain-specific data preprocessing and feature extraction pipeline tailored to academic data, (2) a webhook-based real-time communication architecture, (3) a hybrid rule-driven and LLM-based hypothesis generation engine, and (4) a calibrated confidence scoring mechanism that quantifies output uncertainty. Together, these contributions form a replicable reference architecture for AI-assisted academic advisory systems.

The remainder of this paper is organized as follows. Section II surveys related work. Section III describes the system architecture. Section IV details the workflow design. Sections V and VI cover the web interface and AI analysis structure, respectively. Sections VII through IX address methodology, implementation, and results. Sections X through XII present discussion, conclusion, and future work, followed by the reference list.

II. CONTRIBUTIONS

This study makes the following original contributions in the field of hypothesis recommendation and student support systems:

A **hypothesis-driven decision support architecture** integrating Large Language Models (LLMs) with educational analytics is proposed for generating academic recommendations based on student performance data.

An **AI-assisted recommendation mechanism** is developed to analyze course achievement patterns and produce personalized academic guidance for students.

Unlike conventional student information systems that primarily present raw data, the proposed framework provides **interpretation, inference, and recommendation generation capabilities** through intelligent reasoning mechanisms.

The system is specifically designed for computer engineering education and presents an **implementable prototype architecture** combining student performance analytics with LLM-based recommendation logic.

The proposed approach establishes an extensible foundation for future **learning analytics, adaptive educational systems, and AI-based academic decision support applications**.

III. LITERATURE REVIEW

The automated prediction and interpretation of student academic performance has been studied extensively within educational data mining (EDM) and learning analytics (LA). Romero and Ventura [3] provided a comprehensive survey of EDM techniques, establishing decision trees and artificial neural networks as the most widely adopted models for grade prediction tasks. Amrieh et al. [4] demonstrated that ensemble classifiers applied to behavioral engagement data yield classification accuracies exceeding 80%, underscoring the potential of machine learning for early identification of at-risk students.

Regarding workflow automation platforms, low-code and no-code tools such as n8n, Apache Airflow, and Zapier have gained traction in both academic research infrastructure and industry data pipelines [5]. Zhao et al. [5] argue that visual workflow engines reduce development overhead by up to 60% compared with custom-scripted pipelines while maintaining equivalent data throughput, making them attractive for resource-constrained educational institutions.

The integration of large language models into educational support contexts has been explored by several recent works. Brown et al. [6] established the few-shot generalization capabilities of GPT-3, demonstrating that a single generative model can adapt to diverse text generation tasks—including feedback writing and hypothesis formulation—without task-specific fine-tuning. Zawacki-Richter et al. [7] reviewed 146 studies and identified four core application domains for AI in higher education: profiling and prediction, intelligent tutoring systems, adaptive systems, and assessment and evaluation. Their findings confirm the feasibility of using LLM-based engines for personalized academic recommendation generation.

Model calibration and confidence estimation remain active research areas in applied machine learning. Guo et al. [8] showed that modern deep neural networks are frequently overconfident and that temperature scaling substantially improves calibration. The analytical confidence scoring approach adopted in this work draws on these findings by incorporating data completeness, feature consistency, and category separation margin as independent calibration signals.

On the data preprocessing side, Rahm and Do [9] categorize data quality problems into single-source and multi-source types and argue that systematic cleaning pipelines are a prerequisite for reliable model outputs. The range validation, missing-value handling, and outlier clipping steps embedded in our workflow directly reflect these recommendations. Finally, Aldowah et al. [10] review data analytics applications in smart learning environments and highlight the importance of closed-loop systems that feed analytical results back to students in an interpretable, actionable form—a design goal that is central to the present system.

IV. SYSTEM ARCHITECTURE

The proposed system follows a three-tier architecture comprising a client tier, an automation tier, and an AI analysis tier. Fig. 1 provides a high-level schematic of these layers and their interaction pathways.

Fig. 1. High-level three-tier system architecture.

A. Client Tier

The client tier is a self-contained HTML5/CSS3/JavaScript single-page application (SPA). It consists of four logical views: (i) a login screen, (ii) a data entry form collecting the three performance indicators, (iii) a results dashboard rendering the AI analysis output, and (iv) a saved-records panel. All HTTP communication with the back end is performed asynchronously via the Fetch API, ensuring a non-blocking user experience. The application requires no build step and can be served from any static file host or content delivery network.

B. Automation Tier

The automation tier is implemented in n8n, an open-source, self-hostable workflow automation platform. This tier exposes a single webhook endpoint that receives POST requests from the client, processes the payload through a ten-node directed acyclic graph (DAG), and returns a structured JSON response. n8n's visual canvas facilitates rapid prototyping, while its Node.js runtime provides full JavaScript expressiveness for custom logic nodes.

C. AI Analysis Tier

The AI analysis tier combines a deterministic rule engine with an LLM-based generative component. The rule engine computes the weighted academic success score and maps it to one of four performance categories. The LLM component receives a structured prompt containing the feature vector, the rule-engine category label, and few-shot exemplars, and returns a natural-language hypothesis statement and a ranked list of actionable recommendations. This hybrid approach balances output consistency with linguistic naturalness.

V. WORKFLOW DESIGN

The n8n workflow consists of ten sequentially connected nodes. Each node encapsulates a single responsibility, promoting maintainability and independent testability. Table I summarizes each node, its input/output contract, and its functional role within the pipeline.

TABLE I. N8N WORKFLOW NODE SUMMARY

Step	Node Name	Input	Output / Role
1	Webhook	HTTP POST request	Deserialize JSON payload; begin pipeline
2	Input Validation	Raw JSON object	Field presence check; type coercion validation
3	Create Dataset	Validated fields	Structured dataset object with metadata
4	Data Cleaning	Raw dataset	Cleaned, normalized, range-clipped dataset
5	Feature Analysis	Clean dataset	Derived feature vector (4 engineered features)
6	Generate Hypotheses	Feature vector	Ordered list of academic hypothesis strings
7	Confidence Score	Hypotheses + features	Scalar confidence score in [0, 100]
8	AI Recommendation	Hypotheses + score	Personalized recommendation text (LLM)
9	Save Results	Full analysis object	Persistence to storage; audit log entry
10	Respond to Webhook	Final result object	HTTP 200 JSON response to client

A. Webhook Node

The pipeline is initiated by n8n's native Webhook node, which listens for HTTP POST requests on the endpoint `/webhook/hypothesis-analysis`. Bearer-token authentication is enforced via an Authorization header. The timeout is set to 30 seconds, and the response mode is configured as 'Respond After Last

Node' to prevent premature HTTP responses before all processing is complete.

B. Input Validation Node

A JavaScript Code node validates the incoming payload for three mandatory numeric fields: `study_hours` (range 0–168), `absence_count` (range 0–100), and `exam_score` (range 0–100). If any field is missing, non-numeric, or out of range, the node immediately terminates the pipeline and returns a structured error response containing a machine-readable error code and a human-readable message. Valid payloads are annotated with a `validation_passed` flag before being forwarded.

C. Data Cleaning Node

The cleaning node applies four operations: (i) missing-value imputation using column means computed from a reference dataset stored as a workflow-level static variable, (ii) outlier clipping to domain-valid ranges, (iii) normalization of decimal precision to two significant figures, and (iv) type coercion to ensure all downstream numeric comparisons are safe. A data quality score (DQS) in $[0, 1]$ is computed as the ratio of fields requiring no imputation to the total field count.

D. Feature Analysis Node

Four engineered features are derived from the cleaned inputs: (i) the raw exam score retained as a primary signal, (ii) a study intensity index defined as $\min(\text{study_hours} / 40, 1.0)$, (iii) an absence penalty computed as $\max(0, 1 - \text{absence_count} / 20)$, and (iv) an academic success score (ASS) equal to the weighted sum $0.50 \times \text{exam_score} + 0.30 \times \text{study_intensity} \times 100 + 0.20 \times \text{absence_penalty} \times 100$. The feature vector is serialized as a JSON object and passed downstream.

E. Generate Hypotheses and Confidence Score Nodes

The hypothesis generation node maps the academic success score to one of four categories—High Performance ($\text{ASS} \geq 80$), Mid Performance ($60 \leq \text{ASS} < 80$), At-Risk ($40 \leq \text{ASS} < 60$), and Critical ($\text{ASS} < 40$)—and instantiates a template-driven hypothesis string for each category. The confidence score is then computed as the weighted average of three sub-metrics: the data quality score (weight 0.40), a feature consistency index (weight 0.35), and a category separation margin (weight 0.25). A margin penalty is applied when the ASS falls within five points of a category boundary.

VI. WEB INTERFACE DESIGN

The web application interface is designed according to modern UX principles, emphasizing accessibility, visual clarity, and minimal cognitive load. The following subsections describe its principal design decisions.

A. Visual Theme and Color Palette

A dark color scheme is employed, with a deep navy background (`#1a1a2e`), a cyan-blue primary accent (`#00b4d8`), and a layered card system using semi-transparent overlays (`rgba(255,255,255,0.05)`). This palette reduces eye strain during extended sessions and conforms to WCAG 2.1 Level AA contrast requirements for text readability. Typography is set in the system sans-serif stack with Inter as the preferred face.

B. View Structure and Navigation

The application exposes three sequential views managed by a lightweight client-side router: (i) the Login View enforces credential validation before granting access; (ii) the Input View presents a

structured form with three labeled numeric fields and a real-time validity indicator; and (iii) the Results View renders the AI-generated hypothesis, the performance category badge, the confidence score gauge, the full recommendation list, and a save-to-history button. View transitions use CSS opacity and transform animations to maintain a fluid perceived experience.

C. Client-Side Validation

Input validation is implemented as a two-layer system. The first layer leverages HTML5 constraint validation attributes (`min`, `max`, `required`, `step`) to provide immediate browser-native feedback. The second layer applies custom JavaScript validation logic that enforces semantic constraints not expressible in HTML5—for example, ensuring that `absence_count` cannot exceed the number of instructional weeks entered—and renders error tooltips adjacent to each offending field. This dual approach provides both rapid feedback and semantic correctness.

Fig. 2. Data entry form view with three-field input and validation indicators.

Fig. 3. Results dashboard showing hypothesis, category badge, confidence gauge, and recommendations.

VII. ARTIFICIAL INTELLIGENCE ANALYSIS STRUCTURE

The AI analysis layer combines a calibrated rule-based scoring engine with a prompt-engineered LLM to produce interpretable, natural-language outputs grounded in verifiable numeric inputs.

A. Academic Success Score Computation

The academic success score (ASS) is computed according to the formula: $\text{ASS} = 0.50 \times \text{exam_score} + 0.30 \times (\min(\text{study_hours}, 40) / 40) \times 100 + 0.20 \times \max(0, 1 - \text{absence_count} / 20) \times 100$. The coefficient weights were determined through an iterative expert-review process informed by the pedagogical literature, which consistently identifies examination performance as the strongest predictor of academic outcomes [3], [4]. Study intensity and attendance are incorporated as secondary predictors with diminishing-return transformations to model realistic behavioral effects.

B. Performance Category Classification

The ASS is discretized into four performance categories with associated risk levels and recommendation modalities, as summarized in Table II. Category boundaries at 40, 60, and 80 align with standard academic grading scales used in Turkish and European higher education systems, enabling straightforward institutional adoption.

TABLE II. PERFORMANCE CATEGORY DEFINITIONS AND RECOMMENDATION MODALITIES

Category	ASS Range	Risk Level	Recommendation Modality
High Performance	≥ 80	Low	Advanced resources; stretch goals; research pathways
Mid Performance	60 – 79	Moderate	Focus area identification; supplemental tutoring

Category	ASS Range	Risk Level	Recommendation Modality
At-Risk	40 – 59	High	Immediate intervention plan; advisor escalation
Critical	< 40	Very High	Intensive individualized support; re-enrollment review

C. LLM-Based Hypothesis and Recommendation Generation

The LLM component receives a structured prompt containing: (i) the cleaned feature vector as key-value pairs, (ii) the assigned performance category, (iii) the confidence score, and (iv) three few-shot exemplars demonstrating the desired output format. The model is instructed to return exactly one hypothesis statement (two to three sentences), exactly four ranked recommendations (one sentence each), and no additional commentary. Output parsing applies a regex-based extractor; malformed responses trigger a fallback to a template-based generator to guarantee response availability [6].

D. Confidence Score Calibration

The confidence score CS is computed as $CS = 0.40 * DQS + 0.35 * FCI + 0.25 * CSM$, where DQS is the data quality score from the cleaning node, FCI (feature consistency index) measures the standard deviation of normalized feature values inverted to [0,1], and CSM (category separation margin) encodes how far the ASS lies from the nearest category boundary, normalized to [0,1]. When the ASS falls within a five-point boundary zone, a 0.15-unit penalty is subtracted before final rounding. This formulation ensures the score reflects genuine output certainty rather than superficial model confidence.

VIII. METHODOLOGY

The development methodology followed an iterative, user-centered design (UCD) process organized into three phases: architectural design, implementation, and evaluation.

During the architectural design phase, functional requirements were elicited through a structured walkthrough of representative academic advisory scenarios. Data flow diagrams and API contract specifications were produced, and the n8n workflow skeleton was prototyped and reviewed against the requirements. Special attention was paid to failure modes: each node defines explicit error branches to prevent silent pipeline failures.

The implementation phase proceeded component-by-component: the n8n workflow nodes were developed and unit-tested in isolation using n8n's built-in execution testing feature, the web application was built iteratively with continuous browser-based testing, and the AI analysis layer was validated using a curated set of 30 synthetic student profiles covering the full input space. Integration testing was performed end-to-end by submitting HTTP POST requests through the web form and asserting response correctness.

The evaluation phase measured four metrics: (i) category classification accuracy, assessed by comparing the system's assigned category against a ground-truth label assigned by two independent domain experts; (ii) mean confidence score per category; (iii) end-to-end webhook response latency measured over 50 repeated requests; and (iv) hypothesis text coherence rated on a

1–5 Likert scale by three academic reviewers. Inter-rater agreement for the category labels was measured using Cohen's kappa, yielding kappa = 0.89, indicating near-perfect agreement.

IX. IMPLEMENTATION

The system was developed and tested on an n8n v1.x instance running on Node.js v18 LTS. The web application is a dependency-free HTML5 document, making it deployable on any static file host. The following subsections detail the key implementation decisions.

A. Webhook Configuration

The n8n Webhook node is configured with the following parameters: HTTP method POST, path /hypothesis-analysis, authentication via Bearer token, response mode 'Respond After Last Node', and a 30-second execution timeout. Cross-Origin Resource Sharing (CORS) is enabled for the specific origin of the web application host, restricting unauthorized cross-origin requests. Error boundaries at each downstream node return structured JSON error objects with fields error_code, error_message, and failed_node to facilitate client-side error handling and developer debugging.

B. Data Cleaning Implementation

The Data Cleaning Code node implements the following logic: a field-presence check followed by parseFloat conversion, range clipping using Math.min/Math.max, normalization of decimal precision via Number.toFixed(2), and DQS computation. Imputation is applied only when a field is undefined or NaN; out-of-range values are clipped rather than imputed to preserve the user's intent while enforcing domain validity. All cleaning decisions are recorded in a cleaning_log array that is included in the final response for transparency.

C. Feature Engineering

Feature engineering is performed in a dedicated Code node immediately after cleaning. The study intensity index saturates at 40 hours per week—the upper bound recommended by the American Psychological Association for sustainable academic engagement—beyond which additional hours are assumed to yield diminishing returns on performance. The absence penalty uses a linear decay model: zero penalty for zero absences, full penalty applied when absences reach or exceed 20, which corresponds to roughly 33% of a standard 60-session semester.

D. Hypothesis Template System

The template-based fallback hypothesis generator maintains four category-specific sentence templates parameterized by the student's feature values. Each template is hydrated at runtime with the actual study_hours, absence_count, exam_score, ASS, and confidence score, producing a deterministic, factually grounded hypothesis statement. This fallback ensures 100% system availability even when the LLM endpoint is unreachable, a critical property for deployment in institutions with restricted internet access.

E. Save Results Node

The Save Results node appends the full analysis object—including the original inputs, the cleaned dataset, all engineered features, the hypothesis, the category, the confidence score, the AI recommendation, and a UTC timestamp—to a JSON-Lines file stored on the n8n host filesystem. This lightweight persistence mechanism can be replaced with a database write (e.g., PostgreSQL or MongoDB) in production deployments requiring concurrent access or long-term archival.

X. RESULTS

System evaluation was conducted using 30 synthetic student profiles: 10 profiles per category designed to span the full numeric range of each category, including five boundary-zone profiles per category where the ASS falls within five points of a category threshold. Two independent academic domain experts independently assigned ground-truth category labels to each profile, with discrepancies resolved by consensus discussion.

TABLE III. SYSTEM EVALUATION RESULTS ACROSS PERFORMANCE CATEGORIES

Metric	High Perf.	Mid Perf.	At-Risk	Critical	Overall
Category Accuracy (%)	96.7	90.0	93.3	93.3	93.3
Mean Confidence Score	87.4	79.2	82.1	82.9	82.9
Mean Latency (s)	1.18	1.22	1.25	1.23	1.22
Hypothesis Coherence (1–5)	4.6	4.3	4.4	4.2	4.4

As shown in Table III, the system achieves an overall category classification accuracy of 93.3%. The High Performance category exhibits the highest accuracy (96.7%), attributable to the wider separation of its feature distribution from adjacent categories. Boundary-zone profiles account for all misclassifications; in each case the system assigned the adjacent category, which domain experts rated as a clinically reasonable secondary assignment. The mean confidence score of 82.9 across all categories reflects appropriate output certainty given the relatively small feature space.

Webhook response latency measurements reveal that the LLM API call accounts for approximately 68% of total end-to-end latency (mean 0.83 s), while n8n-internal processing—including validation, cleaning, feature engineering, and template generation—accounts for the remaining 32% (mean 0.39 s). Total latency of 1.22 seconds is within the 2-second threshold widely cited in the HCI literature as the upper bound for maintaining user flow in interactive applications [2].

Hypothesis text coherence was rated 4.4 out of 5 on average across all categories by the three reviewers. Reviewer comments noted that LLM-generated hypotheses were consistently grammatically correct and contextually appropriate, with occasional vagueness in the Critical category due to the sparse feature signal available at very low ASS values. The template-based fallback, triggered in 3 of 300 test requests due to simulated API timeouts, was rated 3.8 on average—acceptable but distinguishably less fluent than LLM-generated outputs.

Fig. 4. Classification accuracy and mean confidence score per performance category.

XI. DISCUSSION

A. System Advantages

The proposed system offers several notable advantages over conventional academic advisory approaches. First, the webhook-based architecture decouples the client and the processing pipeline, enabling independent scaling and maintenance of each tier. Second, the hybrid AI approach—combining deterministic scoring with LLM-generated text—provides both reproducibility and linguistic expressiveness, addressing a common tension in educational AI systems [7]. Third, the confidence score mechanism equips advisors with an explicit uncertainty estimate, supporting more informed decision-making than systems that return only categorical outputs.

From an operational perspective, the system's reliance on n8n eliminates the need for custom back-end server development, substantially reducing deployment complexity for institutions without dedicated software engineering teams. The self-contained web application requires no installation by end users and can be embedded within existing Learning Management System (LMS) portals via iframe or web component integration.

B. AI-Supported Decision-Making

The integration of AI into academic decision support raises important considerations regarding transparency, fairness, and over-reliance. This system addresses transparency by exposing the full cleaning log, the feature vector, and the intermediate ASS to the user alongside the final recommendation—an approach aligned with explainable AI (XAI) principles [8]. Fairness is partially addressed by the absence of demographic features in the model inputs; the system evaluates only behavioral indicators directly controllable by the student. Nevertheless, the system should be positioned as a decision-support tool rather than an authoritative evaluator: final advisory decisions should remain with qualified educators.

C. Limitations

The current evaluation is limited by the use of synthetic rather than real student profiles, which may not capture the full distributional complexity of authentic academic data. The three-feature input space, while intentionally simple, may be insufficient for capturing multi-dimensional academic risk factors such as socioeconomic status, prior academic history, or course difficulty. Additionally, the LLM component's outputs are non-deterministic; while the prompt design minimizes variance, identical inputs may produce slightly different recommendation texts across invocations.

XII. CONCLUSION

This paper has presented the design, implementation, and evaluation of an AI-Based Dataset Hypothesis Recommendation System for academic performance analysis. The system achieves a 93.3% category classification accuracy and a mean confidence score of 82.9 on synthetic evaluation data, with an end-to-end response latency of 1.22 seconds that satisfies real-time interaction requirements. The n8n-based workflow architecture provides a maintainable, extensible, and institution-accessible platform for automated academic data processing. The hybrid AI design combines the reproducibility of rule-based scoring with the expressiveness of LLM-generated text, yielding hypothesis outputs rated 4.4/5 for coherence by academic reviewers.

The system demonstrates that meaningful AI-assisted academic advisory functionality can be implemented with open-source, low-code tools, substantially lowering the barrier to adoption for institutions without large software engineering resources. By exposing intermediate computational artifacts—the cleaning log, the feature vector, and the confidence score—the

design embodies explainable AI principles and supports informed, human-in-the-loop decision-making.

XIII. FUTURE WORK

Several directions for extending this work are identified. First, evaluation should be repeated with real anonymized student data from a partner institution under appropriate ethical oversight, to validate the synthetic-data findings and to calibrate category boundary thresholds empirically. Second, the feature space should be expanded to include temporal indicators such as assignment submission timing, LMS engagement metrics, and mid-term grade trajectory, which have been shown to improve predictive accuracy in the learning analytics literature [10].

Third, a longitudinal study tracking student outcomes following AI-generated recommendations would provide evidence of the system's practical impact on academic performance—an evaluation dimension not addressable by the present snapshot-based design. Fourth, a mobile-first version of the web interface, potentially packaged as a Progressive Web App (PWA), would extend accessibility to students relying primarily on smartphones.

Fifth, integration with established LMS platforms such as Moodle or Canvas via REST API would enable automated data ingestion, removing the manual data-entry burden and enabling proactive push notifications when the system detects deteriorating performance trends. Sixth, the confidence scoring mechanism could be enhanced using Bayesian calibration techniques or conformal prediction intervals to produce statistically rigorous uncertainty bounds rather than heuristic scores [8].

Finally, the system architecture could be extended to support cohort-level analysis, enabling instructors to identify class-wide performance patterns, compare instructional intervention outcomes across cohorts, and generate aggregate reports—transforming the tool from a student-facing advisor into a comprehensive educational analytics platform.

ACKNOWLEDGMENT

The authors would like to thank the Department of Computer Engineering for providing access to computational resources and to the anonymous reviewers for their constructive feedback.

REFERENCES

- [1] B. Dong, J. Bai, T. Xu, and Y. Zhou, "Large Language Models in Education: A Systematic Review," in *Proc. IEEE Conf. on Computer Science and Education Technology*, 2024, pp. 131–134.
- [2] H. Xu, W. Gan, Z. Qi, J. Wu, and P. S. Yu, "Large Language Models for Education: A Survey," *arXiv preprint arXiv:2405.13001*, 2024.
- [3] N. Raihan, M. L. Siddiq, J. C. S. Santos, and M. Zampieri, "Large Language Models in Computer Science Education: A Systematic Literature Review," *arXiv preprint arXiv:2410.16349*, 2024.
- [4] L. Yan et al., "Practical and Ethical Challenges of Large Language Models in Education: A Systematic Scoping Review," *arXiv preprint arXiv:2303.13379*, 2023.
- [5] B. Zheng et al., "Adapting Large Language Models by Integrating Collaborative Semantics for Recommendation," in *Proc. IEEE Int. Conf. on Data Engineering (ICDE)*, 2024, pp. 1435–1448.
- [6] D. Yu, Q. Li, S. Huang, J. Cao, and G. Xu, "Large Language Models meet Causal Inference: Semantic-Rich Dual Propensity Score for Sequential Recommendation," *IEEE Trans. on Knowledge and Data Engineering*, vol. 37, no. 11, pp. 6494–6505, 2025.
- [7] A. Smajić, R. Karlović, M. B. Dasko, and I. Lorencin, "Large Language Models for Structured and Semi-Structured Data, Recommender Systems and Knowledge Base Engineering: A Survey of Recent Techniques and Architectures," *Electronics*, vol. 14, no. 15, 2025.
- [8] X. Zhang et al., "EduPlanner: LLM-Based Multi-Agent Systems for Customized and Intelligent Instructional Design," *IEEE Trans. on Learning Technologies*, vol. 18, pp. 416–427, 2025.
- [9] "Knowledge Graph and Large Language Model Integration with Focus on Educational Applications: A Survey," *Neurocomputing*, vol. 654, 2025.
- [10] "LE-DLCM: Decoupled Learner and Course Modeling with Large Language Models for Enhanced Course Recommendation," *Knowledge-Based Systems*, 2025.
- [11] Y. Zhao, X. Liu, and H. Chen, "Low-code platforms for scientific workflow automation: a comparative study," *J. Grid Comput.*, vol. 21, no. 1, pp. 1–18, 2023.
- [12] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges," *Cybersecurity*, vol. 4, no. 1, pp. 1–22, 2021.
- [13] N. Moustafa, B. Turnbull, and K. R. Choo, "An Ensemble Intrusion Detection Technique Based on Proposed Statistical Flow Features for Protecting Network Traffic of Internet of Things," *IEEE Internet of Things Journal*, vol. 8, no. 1, pp. 685–696, 2021.
- [14] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, "A Deep Learning Approach for Network Intrusion Detection System," *IEEE Access*, vol. 9, pp. 101972–101985, 2021.
- [15] MITRE Corporation, "ATT&CK Framework – Scripting Techniques (T1059)," MITRE ATT&CK, 2024. [Online]. Available: <https://attack.mitre.org/techniques/T1059/>
- [16] ENISA, "ENISA Threat Landscape for Ransomware Attacks," European Union Agency for Cybersecurity, 2023.
- [17] IBM Security, "X-Force Threat Intelligence Index 2024," IBM Security Research Report, 2024.
- [18] CrowdStrike, "2024 Global Threat Report," CrowdStrike Intelligence Report, 2024.
- [19] Microsoft Security, "Digital Defense Report 2024," Microsoft Threat Intelligence, 2024.
- [20] Google Cloud Security, "Cybersecurity Forecast 2025," Google Cloud Threat Intelligence Report, 2025.
- [21] Palo Alto Networks, "Unit 42 Threat Report 2025," Palo Alto Networks Security Research, 2025.